

A Project Report on

**Blockchain-Based Password Manager with Key
Authentication**

Submitted by

Candidate Names

K.RAJASRI

AP24122050006

K.ANUGNA SAI

AP24122050013



CYS 508

BLOCKCHAIN TECHNOLOGY AND APPLICATIONS

SRM University–AP
Neerukonda, Mangalagiri, Guntur
Andhra Pradesh – 522 240
April 2025

Abstract:-

In an increasingly digital world, managing passwords securely has become both critical and challenging. Most traditional password management systems are centralized, creating a single point of failure that hackers can exploit. This project presents a decentralized, blockchain-based password management solution that leverages Ethereum smart contracts and key-based user authentication to enhance security. The system allows users to securely store, view, and delete hashed credentials on a local blockchain network. It integrates dual-layer security through local login verification and key-based authentication for accessing stored data. Built using Python, Web3.py, and Hardhat for blockchain simulation, this solution offers immutability, transparency, and resilience against centralized data breaches. The system is particularly useful in scenarios where password integrity and privacy are paramount.

Introduction:-

In the modern digital ecosystem, individuals and organizations rely heavily on online services, each protected by user credentials—most commonly, usernames and passwords. With the increasing number of accounts one must manage, users tend to reuse passwords or opt for simple, easily guessable combinations. This behavior significantly raises the risk of unauthorized access, identity theft, and large-scale data breaches.

Traditional password managers attempt to address this issue by storing and encrypting passwords in a centralized repository. While encryption provides a layer of security, centralization introduces a fundamental weakness: a single point of failure. If the central server or database is compromised, the attacker gains access to the credentials of all users.

To mitigate this risk, our project proposes a **Blockchain-Based Password Manager with Key Authentication**. This solution harnesses the decentralized and immutable nature of blockchain technology, particularly Ethereum, to store user credentials in a

tamper-proof, distributed ledger. The approach ensures that no central authority holds all the data, significantly reducing the likelihood of mass data exposure.

What sets this system apart is its **multi-layered security model**. Before users can interact with the blockchain or access any stored credentials, they must pass a **local login authentication**. This prevents unauthorized usage of the interface. Additionally, when users attempt to **view their saved passwords**, they are prompted to enter a **secret key** (stored as a hashed constant in the backend). This dual-authentication model reinforces user privacy and access control, ensuring that even if someone gains access to the application interface, stored passwords remain inaccessible without the correct key.

Passwords themselves are **never stored in plaintext**. Instead, the system hashes them using the **SHA-256 algorithm** before they are committed to the blockchain. This ensures that even if blockchain data is publicly accessible, no sensitive information is exposed in readable form.

This system is developed using **Python** for application logic, **Web3.py** for smart contract interaction, and **Hardhat** to simulate a local Ethereum environment. The integration of these technologies results in a secure, transparent, and user-friendly password management platform, ideal for privacy-conscious users.

Tools and Technologies Used:-

1. Python

- Handles command-line interface logic.
- Manages blockchain interactions using Web3.py.
- Facilitates secure login and input handling.
- Easily integrates with cryptographic libraries.
- Offers wide support for testing and debugging tools.

2. Web3.py

- Enables reading smart contract state (e.g., stored passwords).
- Allows transaction signing and broadcasting on Ethereum.
- Supports multiple Ethereum clients (e.g., Hardhat, Geth).
- Maintains compatibility with various blockchain networks.
- Provides seamless event and transaction tracking.

3. Hardhat

- Simulates Ethereum blockchain locally for rapid testing.
- Provides real-time logging of transactions and events.
- Automatically deploys smart contracts on startup.
- Supports automated contract testing and scripting.
- Enables fast resets for testing contract logic repeatedly.

4. Solidity

- Structures data storage using structs and arrays.
- Implements security-focused design (e.g., index-based deletion).
- Ensures all operations are on-chain and auditable.
- Compatible with major Ethereum dev tools (Hardhat, Truffle).
- Enables modular and upgradable contract development.

5. SHA-256 (via hashlib)

- One-way encryption ensures passwords cannot be reversed.
- Offers high speed and collision resistance.
- Used in blockchain mining and security applications.
- Compatible across platforms and languages
- Provides deterministic outputs for consistent hashing.

6. getpass (Python module)

- Prevents passwords from being logged in terminal history.
- Adds security when entering sensitive information like master keys.
- Built-in and requires no external dependencies.
- Commonly used in CLI-based login systems.
- Helps enforce best practices in terminal applications.

7. Ethereum (Local Network)

- Stores password data immutably through smart contracts.
- Ensures decentralized, tamper-resistant infrastructure.
- Eliminates reliance on centralized databases.
- Facilitates traceable transaction history.
- Supports account-based data segregation.

Steps to run the project:-

1. Set Up the Hardhat Blockchain

1.1 Initialize Hardhat Project

In your project folder:

- `mkdir password_manager_blockchain`
- `cd password_manager_blockchain`
- `npm init -y`
- `npm install --save-dev hardhat`
- `npx hardhat`

Select:

- **Create a basic sample project** → Yes for sample project → Install dependencies → Finish.

1.2 Create Smart Contract

Inside Contracts/PasswordManager.sol, create your smart contract:

```
// SPDX-License-Identifier: MIT
```

```
pragma solidity ^0.8.0;
```

```
contract PasswordManager {
```

```
    struct PasswordEntry {
```

```
        string website;
```

```
    string encryptedPassword;  
}
```

```
mapping(address => PasswordEntry[]) private userPasswords;
```

```
    function storePassword(string memory _website, string memory  
_encryptedPassword) public {
```

```
    PasswordEntry memory newEntry = PasswordEntry({
```

```
        website: _website,
```

```
        encryptedPassword: _encryptedPassword
```

```
    });
```

```
    userPasswords[msg.sender].push(newEntry);
```

```
}
```

```
function getPasswords() public view returns (PasswordEntry[] memory) {
```

```
    return userPasswords[msg.sender];
```

```
}
```

```
function deletePassword(uint index) public {
```

```
require(index < userPasswords[msg.sender].length, "Invalid index");

for (uint i = index; i < userPasswords[msg.sender].length - 1; i++) {

    userPasswords[msg.sender][i] = userPasswords[msg.sender][i + 1];

}

userPasswords[msg.sender].pop();

}
```

1.3 Compile the Contract

`npx hardhat compile`

2. Deploy the Contract

2.1 Create Deployment Script

Inside `scripts/deploy.js` ,write:

```
async function main() {

    const PasswordManager = await ethers.getContractFactory("PasswordManager");

    const passwordManager = await PasswordManager.deploy();
```



```
// Wait until the contract is actually deployed

await passwordManager.waitForDeployment();

console.log(`Contract deployed to: ${passwordManager.target}`);
}

// Handle errors properly

main().catch((error) => {

  console.error(error);

  process.exitCode = 1;

});
```

2.2 Start Hardhat Local Blockchain Node

In another terminal:

```
npx hardhat node
```

Hardhat will run at <http://127.0.0.1:8545>.

You'll see accounts listed (these are default accounts you can use).

2.3 Deploy Contract to Local Blockchain

Now in the first terminal:

```
npx hardhat run scripts/deploy.js --network localhost
```

It will print the **deployed contract address**, like:

PasswordManager deployed to: 0x5FbDB2315678afecb367f032d93F642f64180aa3

Note down this contract address — you'll use it in your Python app.py

3. Set Up Python App

- Your app.py should contain:
 - ABI of the PasswordManager smart contract
 - The deployed **contract address** (as printed earlier)
 - Blockchain connection URL: <http://127.0.0.1:8545>.
- (You already have this ready — the one you shared with login + key-check)

5. Run the Python App

Finally, just run your app:

```
python app.py
```

```
from web3 import Web3
```

```
import hashlib
```

```
from getpass import getpass # To securely enter the key without revealing it
```

```
# Blockchain connection
```

```
w3 = Web3(Web3.HTTPProvider('http://127.0.0.1:8545'))
```

```
if not w3.is_connected():
```

```
    print("Failed to connect to blockchain.")
```

```
    exit()
```

```
# Direct ABI content (no external file)
```

```
abi = [
```

```
    {
```

```
        "inputs": [{"internalType": "uint256", "name": "index", "type": "uint256"}],
```

```
        "name": "deletePassword",
```

```
        "outputs": [],
```

```
        "stateMutability": "nonpayable",
```

```
        "type": "function"
```

```
    },
```

```
    {
```

```
        "inputs": [],
```

```
        "name": "getPasswords",
```

```
"outputs": [  
  {  
    "components": [  
      {"internalType": "string", "name": "website", "type": "string"},  
      {"internalType": "string", "name": "encryptedPassword", "type": "string"}  
    ],  
    "internalType": "struct PasswordManager.PasswordEntry[]",  
    "name": "",  
    "type": "tuple[]"br/>  }  
,  
  "stateMutability": "view",  
  "type": "function"  
,  
{  
  "inputs": [  
    {"internalType": "string", "name": "_website", "type": "string"},  
    {"internalType": "string", "name": "_encryptedPassword", "type": "string"}  
  ],  
  "name": "storePassword",
```

```
    "outputs": [],  
  
    "stateMutability": "nonpayable",  
  
    "type": "function"  
}  
]
```

```
# Deployed contract address
```

```
contract_address = "0x5FbDB2315678afecb367f032d93F642f64180aa3" # Update if  
needed
```

```
# Connect to contract
```

```
contract = w3.eth.contract(address=contract_address, abi=abi)
```

```
w3.eth.default_account = w3.eth.accounts[0] # Default sender
```

```
# Hardcoded login credentials
```

```
USERNAME = "admin"
```

```
PASSWORD = "admin123"
```

```
# The key to be entered before viewing passwords (store securely in real scenarios)
```

```
USER_KEY = "supersecretkey" # Store this securely (e.g., in an environment variable)
```

```
# Password hashing function
```

```
def hash_password(password):
```

```
    return hashlib.sha256(password.encode()).hexdigest()
```

```
# Blockchain Functions
```

```
def store_credentials(website, password):
```

```
    hashed_password = hash_password(password)
```

```
    tx_hash = contract.functions.storePassword(website,  
hashed_password).transact({'from': w3.eth.default_account})
```

```
    w3.eth.wait_for_transaction_receipt(tx_hash)
```

```
    print("Credentials stored successfully.")
```

```
def view_credentials():
```

```
    # Prompt for the key
```

```
    entered_key = getpass("Enter your key to view passwords: ") # Hidden input
```

```
    if hash_password(entered_key) != hash_password(USER_KEY):
```

```
        print("Invalid key. Exiting...")
```

```
        exit()
```

```
credentials = contract.functions.getPasswords().call()
```

```
if not credentials:
```

```
    print("No credentials stored.")
```

```
else:
```

```
    for idx, entry in enumerate(credentials):
```

```
        print(f"{idx + 1}. Website: {entry[0]}, Hashed Password: {entry[1]}")
```

```
def delete_credentials(index):
```

```
    tx_hash = contract.functions.deletePassword(index).transact({'from':  
w3.eth.default_account})
```

```
    w3.eth.wait_for_transaction_receipt(tx_hash)
```

```
    print("Credential deleted successfully.")
```

```
# Authentication Function
```

```
def login():
```

```
    print("=== Login Required ===")
```

```
    username = input("Enter username: ")
```

```
    password = input("Enter password: ")
```

```
if username == USERNAME and password == PASSWORD:
```

```
print("Login successful!\n")
```

```
else:
```

```
print("Invalid credentials. Exiting...")
```

```
exit()
```

```
# Main Program
```

```
def main():
```

```
    login() # Require login first
```

```
while True:
```

```
    print("\n--- Blockchain Password Manager ---")
```

```
    print("1. Store Credentials")
```

```
    print("2. View Credentials")
```

```
    print("3. Delete Credentials")
```

```
    print("4. Exit")
```

```
    choice = input("Enter your choice: ")
```

```
    if choice == '1':
```

```
        website = input("Enter website: ")
```

```
        password = input("Enter password: ")
```



```
store_credentials(website, password)
```

```
elif choice == '2':
```

```
view_credentials() # This will now require a key before showing passwords
```

```
elif choice == '3':
```

```
view_credentials()
```

```
index = int(input("Enter the index (number) to delete: ")) - 1
```

```
delete_credentials(index)
```

```
elif choice == '4':
```

```
print("Goodbye!")
```

```
break
```

```
else:
```

```
print("Invalid choice. Try again.")
```

```
if __name__ == "__main__":
```

```
main()
```

It will prompt for:

- Login (username + password)
- Then show menu (Store/View/Delete credentials)

Detailed Project Flow:-

The password manager system follows a structured and secure operational flow:

1. User Login Authentication

Before accessing any password management functionality, users must log in.

- Login credentials (username and password) are verified locally.
- Only after successful authentication does the system allow interaction with blockchain functionalities.
- This prevents unauthorized access and adds an extra security layer atop the blockchain.

2. Blockchain Connection Setup

The system connects to a locally running Ethereum blockchain powered by **Hardhat**.

- The smart contract, which manages password storage, is already deployed on this blockchain.
- **Web3.py** is used to establish the connection and interact with the smart contract.

3. Smart Contract Interaction

The smart contract offers three main functions:

Function	Purpose
storePassword	Store the website and encrypted (hashed) password into the blockchain
getPassword	Retrieve all stored credentials associated with the user's blockchain address
deletePassword	Delete a specific stored credential based on its index (position in the stored array)

Each operation creates a blockchain transaction, ensuring every action is traceable and verifiable.

4. Password Management

The password manager performs the following functionalities:

Storing Credentials

- Users enter the website and password.
- Passwords are hashed using the SHA-256 algorithm before being sent to the blockchain.
- Hashing adds an extra layer of security, ensuring plaintext passwords are never exposed.

Viewing Credentials

- Users can fetch and view a list of all websites and their corresponding hashed passwords stored under their account.
- **Key Authentication:** Before viewing the saved passwords, the system requires the user to enter a **key** (which is hashed and compared). This ensures that only authorized users can access their saved credentials.
 - The user must input the correct key (stored as **USER_KEY** in the backend code) to decrypt and view their passwords.

Deleting Credentials

- Users can delete a password entry by selecting its index.
- Internally, the smart contract shifts array elements and removes the last entry to maintain continuity.

5. Logging Out

- Users can exit the system at any point by selecting the appropriate menu option.
- Upon exiting, the blockchain connection is safely terminated.

Output Screen shots:-

The Fig.1 screenshot has command :-

`npx hardhat node:-`

This command starts a **local Ethereum blockchain** using **Hardhat**, a development environment for Ethereum-based projects.

Started HTTP and WebSocket JSON-RPC server at <http://127.0.0.1:8545/>

This means the local Ethereum node is running and ready to accept JSON-RPC requests.

Port 8545 is the default for local development.

Generated Accounts

Hardhat automatically creates **20 unlocked Ethereum accounts**, each with:

- **10,000 ETH** (fake/test ETH only for development).
- **Account Address** (e.g., `0xf39f6de51aad88f...`)
- **Private Key** (e.g., `0xac0974bce391a72...`)

These are **development-only keys** meant to be used locally for testing smart contracts.

```

C:\Users\anugn\OneDrive\Desktop\password_manager_blockchain (1)>npx hardhat node
Started HTTP and WebSocket JSON-RPC server at http://127.0.0.1:8545/

Accounts
=====

WARNING: These accounts, and their private keys, are publicly known.
Any funds sent to them on Mainnet or any other live network WILL BE LOST.

Account #0: 0xf39Fd6e51aad88F6F64ce6aB8827279cFFb92266 (10000 ETH)
Private Key: 0xac0974bec39a17e36ba4a6b4d238ff944bacb478cbed5efcae784d7bf4f2ff80

Account #1: 0x70997970C51812dc3A010C7d01b50e0d17dc79C8 (10000 ETH)
Private Key: 0x59c6995e998f97a5a0044966f094538dc9e86dae88c7a8412f4603b6b78690d

Account #2: 0x3C44CdDdB6a900fa2b585dd299e03d12FA4293BC (10000 ETH)
Private Key: 0x5de4111afa1a4b94908f83103eb1f1706367c2e68ca870fc3fb9a804cdab365a

Account #3: 0x90F79bf6EB2c4f870365E785902E1f101E93b906 (10000 ETH)
Private Key: 0x7c852118294e51e653712a81e05800f419141751be58f605c371e15141b007a6

Account #4: 0x15d34AAf54267DBD7c367833AAf71A00a2C6A65 (10000 ETH)
Private Key: 0x47e179ec197488593b187f80a00eb0da91f1b9d0b13f8733639f19c30a34926a

Account #5: 0x9965507D1a55bcC2695C58ba16FB37d819B0A4dc (10000 ETH)
Private Key: 0x8b3a350cf5c34c9194ca85829a2df0ec3153be0318b5e2d3348e872092edffba

Account #6: 0x976EA74026E726554dB657FA54763abd0C3a0aa9 (10000 ETH)
Private Key: 0x92db14e403b83dfe3df233f83dfa3a0d7096f21ca9b0d6d6b8d88b2b4ec1564e

Account #7: 0x14dc79964da2C08b23698B3D3cc7Ca32193d9955 (10000 ETH)
Private Key: 0x4bbbf85ce3377467afe5d46f804f221813b2bb87f24d81f60f1fcd9f7cbf4356

Account #8: 0x23618e81E3f5cdF7F54C3d65f7FBc0aBf5B21E8f (10000 ETH)
Private Key: 0xdbda1821b80551c9d65939329250298aa3472ba22feea921c0cf5d620ea67b97

Account #9: 0xa0Ee7A142d267C1f36714E4a8F75612F20a79720 (10000 ETH)
Private Key: 0x2a871d0798f97d79848a013d4936a73bf4cc922c825d33c1cf7073dff6d409c6

Account #10: 0xBcd404DE499D14e55001Ccb824a551F3b954096 (10000 ETH)
Private Key: 0xf214f2b2cd398c806f84e317254e0f0b801d0643303237d97a22a48e01628897

```

Fig.1

The below screenshot Fig.2 shows the terminal output of a **Python-based Blockchain Password Manager** application. Here's a breakdown of what is happening in each part of the terminal:

- The application starts by requiring a **login**.
- The user logs in with:
 - **Username:**admin
 - **Password:**admin123

- The system confirms a **successful login**.

After login, the user is shown a **menu** with options to:

1. Store new credentials
2. View existing credentials
3. Delete credentials
4. Exit the program

```
C:\Users\anugn\OneDrive\Desktop\password_manager_blockchain (1)>python frontend/app.py
=== Login Required ===
Enter username: admin
Enter password: admin123
Login successful!

--- Blockchain Password Manager ---
1. Store Credentials
2. View Credentials
3. Delete Credentials
4. Exit
Enter your choice: 2
Enter your key to view passwords:
1. Website: SRM AP, Hashed Password: e955a1a1b722cd127b51ac109f8bd0cab1c9cf0c97f5193c789f3663b994798c
2. Website: , Hashed Password: 2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b9824
3. Website: srmmap, Hashed Password: cef6d58d45e1bce2c48bbf01a783535d4c5e7dadedbe0d0c03c0affce56070a
4. Website: hello, Hashed Password: 8e5ab33caaa579546f90c53645bba1361f8028dda1333d8ca202ec5fb8ed0f4a

--- Blockchain Password Manager ---
1. Store Credentials
2. View Credentials
3. Delete Credentials
4. Exit
Enter your choice: 1
Enter website: NYU Tandon School
Enter password: BMW X7
Credentials stored successfully.

--- Blockchain Password Manager ---
1. Store Credentials
2. View Credentials
3. Delete Credentials
4. Exit
Enter your choice: 2
Enter your key to view passwords:
1. Website: SRM AP, Hashed Password: e955a1a1b722cd127b51ac109f8bd0cab1c9cf0c97f5193c789f3663b994798c
2. Website: , Hashed Password: 2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b9824
3. Website: srmmap, Hashed Password: cef6d58d45e1bce2c48bbf01a783535d4c5e7dadedbe0d0c03c0affce56070a
4. Website: hello, Hashed Password: 8e5ab33caaa579546f90c53645bba1361f8028dda1333d8ca202ec5fb8ed0f4a
5. Website: NYU Tandon School, Hashed Password: 1beb689bdd31aebb73119a8a614294d6920d8b953e72cdc5859e3218260b8e55
```

Fig.2

The Fig.3 These show **two separate write transactions** to the contract function storepassword, which adds password entries:

First storePassword Transaction

- Transaction Hash: 0x80d763d...ce0415a
- From: same user address
- To (Contract Address): 0x5fbd2315678afecb367f032d93f642f64180aa3
- Gas Used: 99435 of 199435
- Mined in Block 5

Second storePassword Transaction:-

- Transaction Hash: 0x35fd908...b4c9f51
- Mined in Block #6
- Gas Used: 115967 of 215958


```

eth_chainId
web3_clientVersion
eth_accounts
eth_getBlockByNumber
eth_chainId (2)
eth_estimateGas
eth_chainId
eth_getBlockByNumber
eth_sendTransaction
  Contract call: PasswordManager#storePassword
  Transaction: 0x80d7638c9d3e70548760d6febcee9c9c06d1ceefbc155bad074c5a82dce0415a
  From: 0xf39fd6e51aad88f6f4ce6ab8827279cfff92266
  To: 0x5fbdb2315678afecb367f032d93f642f64180aa3
  Value: 0 ETH
  Gas used: 99435 of 199435
  Block #5: 0xc8403f0183412e1c368c6404d48f5a57392e920e67946bf8d391934e9f9f15bc

eth_chainId
eth_getTransactionReceipt
eth_getBlockByNumber
eth_chainId (2)
eth_estimateGas
eth_chainId
eth_getBlockByNumber
eth_sendTransaction
  Contract call: PasswordManager#storePassword
  Transaction: 0x35206df84e496176affebb3b75d78ecb8a7314e080c390ef2c62b5908b4c9f51
  From: 0xf39fd6e51aad88f6f4ce6ab8827279cfff92266
  To: 0x5fbdb2315678afecb367f032d93f642f64180aa3
  Value: 0 ETH
  Gas used: 119558 of 219558
  Block #6: 0xb17b15c597f038dbdabe0429cc6bc842b9a5f5349fe34b4c95ca0d5a5677130b

eth_chainId
eth_getTransactionReceipt
eth_chainId
eth_call

```

Fig.3

The Fig.4 shows Read Operations (eth_call)

- These are view functions (i.e., they don't cost gas or alter blockchain state):
- Function getPasswords is called.
- From: 0xf39fd6e51aad88f6f4ce6ab8827279cfff92266
- To (Contract Address): 0x5fbdb2315678afecb367f032d93f642f64180aa3
- It happens multiple times, likely to fetch updated password data after write/delete operations.

Write Operation (eth_sendTransaction)

- This is a state-changing transaction:
- Function deletePassword is called.
- A transaction hash is shown: 0x6a42cdb...f8457

- Gas used: 55228 of 169035
- Block number: #13

```

eth_chainId
eth_getTransactionReceipt
eth_chainId
eth_call
  Contract call: PasswordManager#getPasswords
  From: 0xf39fd6e51aad88f6f4ce6ab8827279cfff92266
  To: 0x5fbdb2315678afecb367f032d93f642f64180aa3

eth_chainId (2)
eth_call
  Contract call: PasswordManager#getPasswords
  From: 0xf39fd6e51aad88f6f4ce6ab8827279cfff92266
  To: 0x5fbdb2315678afecb367f032d93f642f64180aa3

eth_chainId
eth_getBlockByNumber
eth_chainId (2)
eth_estimateGas
eth_chainId
eth_getBlockByNumber
eth_sendTransaction
  Contract call: PasswordManager#deletePassword
  Transaction: 0x6a42cdb97c846119fd7a06631f0d55934dbd406a9180ce8c261fb8c8afff8457
  From: 0xf39fd6e51aad88f6f4ce6ab8827279cfff92266
  To: 0x5fbdb2315678afecb367f032d93f642f64180aa3
  Value: 0 ETH
  Gas used: 55228 of 169035
  Block #13: 0x372819a2f9dd6eb98c959ae552b69f45dcf55a8f74b5af94322af1b935d6dcf8

eth_chainId
eth_getTransactionReceipt
eth_chainId
eth_call
  Contract call: PasswordManager#getPasswords
  From: 0xf39fd6e51aad88f6f4ce6ab8827279cfff92266
  To: 0x5fbdb2315678afecb367f032d93f642f64180aa3

eth_chainId

```

Fig.4

Use Cases and Applicability

The Blockchain-Based Password Manager has broad applicability across various domains. Below are key real-world use cases where this system offers tangible benefits:

1. Individual Password Security

- **Problem:** Most individuals reuse passwords across sites.
- **Solution:** This system allows secure, decentralized storage of hashed passwords with access protected by key authentication.
- **Benefit:** Even if the device or software is compromised, hashed passwords and key protection prevent exposure.

2. Small and Medium Enterprises (SMEs)

- **Problem:** SMEs often lack the budget or infrastructure for enterprise-grade password managers.
- **Solution:** A local, blockchain-powered system requires no third-party subscriptions or internet access.
- **Benefit:** Affordable and secure password management tailored to internal staff.

3. Educational Institutions

- **Problem:** Universities and schools often manage multiple accounts for staff and students.

- **Solution:** Deploy on campus servers using local login and blockchain for auditability.
- **Benefit:** Increases accountability while reducing administrative overhead.

Conclusion

The Blockchain-Based Password Manager offers a secure and decentralized solution for managing personal credentials. By leveraging blockchain's decentralization, immutability, and transparency, it eliminates the risks of centralized data breaches. The system includes local login authentication, hashed password storage, and secret key access, ensuring robust protection against unauthorized access. With simple functions for storing, viewing, and deleting passwords, users retain full control and traceability over their data. This project lays the groundwork for future enhancements in encryption, usability, and cross-network compatibility, promoting safer and more private digital identity management.